

Software Requirements Specification

for

IoT Smart Agriculture Monitoring and Controlling System with AI



Supervisor

Mr. Imran Rasheed

Prepared by

Maqsood Ahmad
22pwbc0924

Zohaib Hassan
22pwbc0935

Asad Ullah
22pwbc0943

Version 1.0 approved

***Department of Computer Science & Information Technology, University of Engineering &
Technology, Peshawar***

Date Created: 20 November 2025

Certificate of Approval

This is to certify that the proposal report for the project titled “**IoT Smart Agriculture Monitoring and Controlling System with AI**” submitted to the Department of Computer Science, UET Peshawar by **Maqsood Ahmad, Zohaib Hassan** and **Asad Ullah** is in the partial fulfillment requirements for the BS (Computer Science).

Signature

Mr. Imran Rasheed
Project Supervisor

Signature

Dr. Wajeeha Khalil
FYP Coordinator

Table of Contents

Table of Contents	
1. Introduction	1
1.1 Purpose	1
1.2 Document Conventions	1
1.3 Intended Audience and Reading Suggestions	1
1.4 Product Scope	2
1.5 References	3
2. Overall Description	3
2.1 Product Perspective	3
2.2 Product Functions	4
2.3 User Classes and Characteristics	6
2.4 Operating Environment	7
2.5 Design and Implementation Constraints	8
2.6 User Documentation	8
2.7 Assumptions and Dependencies	9
3. External Interface Requirements	9
3.1 User Interfaces	9
3.2 Hardware Interfaces	11
3.3 Software Interfaces	11
3.4 Communications Interfaces	12
4. System Features	14
4.1 Feature: Real-Time Sensor Monitoring	14
4.2 Feature: Offline Alerts & Recommendations (Bluetooth Mode)	15
4.3 Feature: Online Advanced Recommendations (AI-assisted)	16
4.4 Feature: Mobile Application	16
4.5 Feature: Cloud Database & Storage	17
4.6 Feature: Security	18
5. Other Nonfunctional Requirements	18
5.1 Performance Requirements	18
5.2 Safety Requirements	19
5.3 Security Requirements	19
5.4 Software Quality Attributes	19
5.5 Business Rules	20

6. Other Requirements	21
6.1 Database Requirements	21
6.2 Internationalization Requirements	21
6.3 Legal and Regulatory Requirements	21
6.4 Environmental Requirements	22
6.5 Future Enhancements	22
6.6 Reusability Requirements	22
Appendix A: Glossary	23
Appendix B: Analysis Models	25
B.1 – Context Diagram (0-Level DFD)	25
B.2 – Level-1 DFD (Decomposition of Main System)	26
B.3 – Use Case Diagram	27
B.4 – System Architecture (Layered Architecture Diagram)	27
B.5 – Entity-Relationship (ER) Diagram (High Level)	28
B.6 – System Workflow Diagram	29
Appendix C: To Be Determined (TBD) List	30

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to formally describe the requirements for the IoT Smart Agriculture Monitoring and Controlling System with AI, developed as a Final Year Project.

This SRS defines the complete functional and non-functional requirements of the system, including:

- IoT-based environmental monitoring for tobacco crops
- Offline Bluetooth-based sync with mobile
- Online cloud-based data storage and analytics
- Basic automation recommendations
- Extendable AI components (optional based on time/resources)

This document will serve as a reference for developers, testers, supervisors, evaluators, and stakeholders throughout the system's development and maintenance.

1.2 Document Conventions

- REQ-XXX format is used for requirement identifiers.
- The terms “shall” and “must” indicate mandatory requirements.
- The terms “should” and “may” indicate optional or recommended requirements.
- All diagrams follow standard notations (DFD, layered architecture).
- Technical terms are defined in Appendix A: Glossary.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following groups:

- Developers: to implement hardware, firmware, mobile app, and backend.
- Supervisors & Evaluators: to track whether requirements meet academic and technical expectations.
- Testers: to prepare system validation and testing plans.
- Agricultural Advisors/Experts: to review assumptions about optimum crop parameters.

Suggested reading order:

1. Overview (Sections 1–2)
2. System interfaces (Section 3)
3. Functional features (Section 4)
4. Nonfunctional requirements (Section 5)
5. Diagrams and glossary (Appendices)

1.4 Product Scope

The system aims to support tobacco farmers by enabling efficient crop monitoring, water usage optimization, and fertilizer planning using low-cost IoT hardware. It provides:

- Real-time sensor monitoring
- Offline (Bluetooth) alerts and recommendations
- Online cloud-based analytics
- Historical data visualization
- Expandable AI-based analysis for improved decision making

The product supports rural agricultural conditions, where internet availability may be limited, making the offline-first approach essential.

1.5 References

- [Harnessing sensors, smart devices, and AI could transform agriculture](#)
- [Integrating artificial intelligence and Internet of Things \(IoT\) technologies for precision agriculture" discusses AI and IoT transforming crop monitoring and farming efficiency](#)
- [Agriculture Monitoring System Using IoT and ML smart and Improved Farming](#)
- [AI in Agriculture-The Future of Farming](#)

2. Overall Description

2.1 Product Perspective

The IoT Smart Agriculture Monitoring and Controlling System with AI is a stand-alone system combining hardware (IoT sensors + ESP32), a mobile application, and a backend server.

The system operates in two modes:

- Offline Mode (Bluetooth)
 - ESP32 communicates directly with the farmer's mobile app.
 - The app receives basic alerts & recommendations without the internet.
 - Suitable for remote fields where connectivity is limited.
- Online Mode (Wi-Fi/Cloud)
 - Sensor data is uploaded to a cloud server.
 - The mobile app retrieves real-time data, analytics, and history.

The system is not a continuation of an existing product; it is a new, self-contained solution, inspired by modern smart-farming architectures but developed independently.

A high-level context view is:

- External Environment: Sensors → ESP32 → Mobile App / Cloud
- Users: Farmers
- Outputs: Alerts, recommendations, irrigation suggestions, data charts

A detailed context diagram will be provided in Appendix B.

2.2 Product Functions

At a high level, the system provides the following core functions:

- Environmental Monitoring
 - Soil moisture, temperature, humidity, PH of the soil.
- Offline Bluetooth Communication
 - Sync and receive alerts without internet
- Cloud Connectivity (Online Mode)
 - Upload sensor readings to server
 - Fetch historical analytics
- Automated Irrigation Recommendations
 - Based on moisture + temperature thresholds
- Fertilizer Planning (Rule-Based)

- Recommendations based on crop stage and guidelines
- Alerting System
 - Low moisture alert
 - Temperature/humidity threshold crossings
- User Dashboard (Mobile App)
 - Real-time values
 - Trend graphs
 - Crop advisory data

Expanded functional requirements will be detailed in Section 4.

2.3 User Classes and Characteristics

Primary User: Farmer

- Technical Skills: Basic smartphone usage
- Needs:
 - Simple UI
 - Offline capability
 - Clear alerts & suggestions
- Language Preference: English / Urdu
- Frequency of use: Daily

Secondary User: System Administrator

- Technical Skills: Intermediate
- Responsibilities:
 - Device pairing
 - Cloud backend management

- Version updates
- Frequency: Occasional

2.4 Operating Environment

The system will operate in:

- Hardware: ESP32 microcontroller with sensors
- Mobile App: Android (Android 7+)
- Connectivity:
 - Bluetooth Low Energy (BLE)
 - Wi-Fi 2.4 GHz for cloud sync
- Server:
 - Cloud database
 - REST API backend (Django / NodeJS, final tech TBD)
- Power: Battery/solar for ESP32 (optional)

2.5 Design and Implementation Constraints

- ESP32 hardware capabilities limit processing and memory
- BLE range 10–40 meters (outdoor)
- Wi-Fi availability inconsistent in target areas
- Use of low-cost sensors due to budget constraints
- Compliance with agriculture domain thresholds
- Mobile app must remain lightweight for low-end devices
- Cloud storage cost must be minimal (student budget)
- Project must be deliverable within academic timeline

2.6 User Documentation

At minimum, the following documentation will be provided:

- User Manual (PDF + in-app help)
- Device Setup Guide (pairing, calibration, mounting)
- Maintenance Guide (sensor care, battery guidance)
- Quick Start Guide (offline + online usage steps)

2.7 Assumptions and Dependencies

Assumptions:

- Farmers have access to an Android phone with Bluetooth.
- The field has at least occasional Wi-Fi or hotspot availability.
- Sensors will be periodically maintained/cleaned.
- Default crop-specific threshold values will be provided by agronomy sources.

Dependencies:

- ESP32 firmware reliability
- Mobile app compatibility
- Cloud API availability
- Sensor accuracy within acceptable error range
- Environmental factors (rainfall, soil type) affecting readings

3. External Interface Requirements

3.1 User Interfaces

The system will provide the following user-facing interfaces:

Mobile Application (Android)

- Dashboard Screen

- Displays real-time sensor readings (soil moisture, temperature, humidity, pH/EC if available).
 - Color indicators (green/yellow/red) for crop-condition status.
- Alerts Screen
 - Shows irrigation alerts, threshold warnings, and fertilizer reminders.
- History & Analytics Screen
 - Graphs for moisture/temperature trends.
 - Filter by date range.
- Device Pairing Screen
 - Connect/disconnect ESP32 via Bluetooth.
 - Wi-Fi setup for online mode.
- Settings Screen
 - Crop selection (tobacco initially).
 - Threshold editing (optional).
 - Language selection (English/Urdu).

UI will follow simple design patterns suitable for low-tech users.

3.2 Hardware Interfaces

The system interacts with the following hardware:

- ESP32 Microcontroller
 - Acts as the core IoT device.
 - Sends sensor data to mobile/ cloud.
- Sensors (Low-Cost Agriculture Sensors)
 - Soil moisture sensor
 - Temperature & humidity (DHT11/22 or similar)
 - Soil pH sensor (optional)

- EC sensor (optional)

Communication between ESP32 and sensors uses standard GPIO, ADC, or I2C depending on sensor type.

3.3 Software Interfaces

- Mobile App ↔ ESP32 Firmware
 - Bluetooth Low Energy (BLE) data packets
 - JSON-formatted measurement data
- Mobile App ↔ Cloud Server
 - REST API (JSON)
 - Endpoints for:
 - POST sensor data
 - GET historical data
 - GET threshold recommendations
 - User authentication (optional)
- Cloud Server Components
 - Database (Firebase / MongoDB / PostgreSQL; final selection TBD)
 - Backend application (Django REST / Node.js Express; TBD)

3.4 Communications Interfaces

Bluetooth (Offline mode)

- Protocol: BLE
- Purpose:
 - Device pairing
 - Data sync
 - Alerts & recommendations

Wi-Fi (Online mode)

- 2.4 GHz Wi-Fi supported by ESP32
- Purpose:
 - Uploading sensor readings
 - Fetching historical data
 - Remote monitoring

Data Formatting

- JSON objects containing:
 - Sensor values
 - Timestamp
 - Device ID

Security

- Encrypted API tokens for server communication
- BLE pairing secured with PIN or pairing handshake (simple level, suitable for farmers)

4. System Features

The system provides multiple core services.
Each feature below includes:

- Description
- Priority
- Stimulus/Response flows
- Functional requirements (REQ-#)

4.1 Feature: Real-Time Sensor Monitoring

Description and Priority

The system collects real-time data from IoT sensors (soil moisture, temperature, humidity, optional pH/EC) and displays it on the mobile app.

Priority: High

Stimulus/Response Sequence

- ESP32 reads sensor values.
- Data is sent to the mobile app (BLE offline) or cloud (WiFi online).
- User opens app → sees latest readings.
- If values exceed defined thresholds, alerts are triggered.

Functional Requirements

- REQ-1: The system shall collect sensor readings at predefined intervals (e.g., every 10–15 minutes).
- REQ-2: The system shall send readings to the mobile app via Bluetooth if offline.
- REQ-3: The system shall upload readings to cloud when Wi-Fi is available.
- REQ-4: The mobile app shall display updated readings instantly.
- REQ-5: The system shall generate alerts when thresholds are exceeded.

4.2 Feature: Offline Alerts & Recommendations (Bluetooth Mode)**Description and Priority**

Farmers receive basic irrigation and threshold-based fertilizer alerts even without internet.
Priority: High (unique feature)

Stimulus/Response Sequence

- Farmer opens app within Bluetooth range.
- ESP32 syncs stored readings via BLE.
- Mobile app compares values to stored optimum data.
- Offline recommendations displayed.

Functional Requirements

- REQ-6: The system shall support Bluetooth syncing when no internet is available.
- REQ-7: The app shall store optimum crop conditions locally.
- REQ-8: The app shall generate irrigation/fertilizer alerts offline using rule-based logic.

- REQ-9: ESP32 shall store at least the last N readings for offline syncing (N = TBD).

4.3 Feature: Online Advanced Recommendations (AI-assisted)

Description and Priority

Advanced analytics (trend detection, smart predictions) work only in online mode.
AI model integration is optional (time & resource dependent).

Priority: Medium

Stimulus/Response Sequence

- Sensor data uploaded to cloud.
- Backend processes values with rule-based logic (initial) or ML model (optional).
- Advanced recommendations sent back to app.

Functional Requirements

- REQ-10: When internet is available, the system shall upload data to the cloud.
- REQ-11: The cloud backend shall process readings and generate detailed recommendations.
- REQ-12: The system shall display advanced analytics (trends, graphs).
- REQ-13: Optional AI models shall improve accuracy of recommendations.

4.4 Feature: Mobile Application

Description and Priority

The mobile app acts as the user interface for monitoring, alerts, device setup, and analytics.
Priority: High

Stimulus/Response Sequence

- User opens the app → views dashboard with sensor values.
- User pairs ESP32 via BLE or connects via Wi-Fi.
- User checks history, graphs, alerts, settings.

Functional Requirements

- REQ-14: The app shall support device pairing via BLE.
- REQ-15: The app shall allow Wi-Fi setup for the ESP32.
- REQ-16: The app shall display history graphs and analytics.
- REQ-17: The app shall allow changing language (English/Urdu).
- REQ-18: The app shall send notifications for critical alerts.

4.5 Feature: Cloud Database & Storage

Description and Priority

The cloud stores historical data for long-term analysis.

Priority: Medium

Functional Requirements

- REQ-19: The system shall store sensor readings with timestamps.
- REQ-20: The backend shall support retrieving historical data.
- REQ-21: The system shall store user/device profiles (optional).

4.6 Feature: Security

Description and Priority

Ensures safe communication between mobile, IoT device, and cloud.

Priority: High

Functional Requirements

- REQ-22: The system shall authenticate users accessing cloud services.
- REQ-23: BLE pairing shall require a secure handshake or PIN.
- REQ-24: API communication shall use encrypted tokens (HTTPS).

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- NFR-1: The system shall update sensor readings within 10–20 seconds after receiving data from ESP32.
- NFR-2: The mobile app shall load the dashboard within 3 seconds on devices with average hardware specifications.
- NFR-3: Bluetooth data transfer shall complete within 5 seconds for each offline sync session (depending on stored data length).
- NFR-4: Cloud database queries (history retrieval, analytics) shall respond within 2–4 seconds under normal load.
- NFR-5: The ESP32 shall operate continuously for long periods and sample sensors at defined intervals without missing readings.

5.2 Safety Requirements

- NFR-6: The system shall not trigger any automatic physical action that could damage crops (e.g., no automatic irrigation pumps; only recommendations).
- NFR-7: Any hardware wiring must follow safe voltage/current ratings to avoid electrical hazards in the field.
- NFR-8: The system shall ensure no unsafe recommendations are generated (e.g., no extreme nutrient suggestions outside standard boundaries).

5.3 Security Requirements

- NFR-9: Communication with cloud APIs shall use HTTPS encryption.
- NFR-10: Bluetooth pairing shall use a secure authentication mechanism (PIN/UUID).
- NFR-11: Only authenticated users shall access their device data in the cloud.
- NFR-12: The system shall protect confidential data (field readings, farmer information) from unauthorized access.
- NFR-13: User passwords shall be stored using secure hashing mechanisms.

5.4 Software Quality Attributes

- Usability:

- NFR-14: The app interface shall be simple, farmer-friendly, and support English/Urdu.
- Reliability:
 - NFR-15: The system shall maintain >95% uptime during active testing.
- Maintainability:
 - NFR-16: Codebase shall be modular to allow updates (adding more sensors, crops, or features).
- Scalability:
 - NFR-17: The backend shall support multiple registered devices and farms.
- Portability:
 - NFR-18: The app shall run on modern Android devices (Android 8+).
- Extensibility:
 - NFR-19: Architecture shall allow future integrations like disease detection, more sensors, or automation.

5.5 Business Rules

- NFR-20: The system shall provide recommendations only, not enforce automated decisions.
- NFR-21: Only authorized users (farmer/owner) can modify optimum crop condition settings.
- NFR-22: Offline recommendations must always be consistent with stored optimum values.

6. Other Requirements

6.1 Database Requirements

- REQ-DB-1: The system shall maintain a structured database storing sensor readings, user profiles, farm profiles, and optimum crop conditions.
- REQ-DB-2: The database shall support both local storage (SQLite on mobile) for offline mode and cloud storage for online mode.

- REQ-DB-3: The system shall synchronize local and cloud databases when the internet becomes available.
- REQ-DB-4: Each ESP32 device shall be linked to a unique farmer account.

6.2 Internationalization Requirements

- REQ-I18N-1: The application shall support English by default.
- REQ-I18N-2: The system architecture shall allow adding additional languages (such as Urdu or Pashto) in future iterations without major redesign.
- REQ-I18N-3: All interface strings shall be stored separately from logic for easy translation.

6.3 Legal and Regulatory Requirements

- REQ-LR-1: The system shall comply with local data privacy practices (no unnecessary storage of personal data).
- REQ-LR-2: All collected field data belongs to the farmer; the system will not share it with any third party.
- REQ-LR-3: Hardware used (ESP32, sensors) shall operate within safe low-power voltage levels to avoid field hazards.

6.4 Environmental Requirements

- REQ-ENV-1: The hardware enclosure shall protect internal components from dust and moisture.
- REQ-ENV-2: All hardware components shall tolerate typical outdoor temperature ranges in agricultural regions.

6.5 Future Enhancements

These are not part of the current SRS scope, but the system shall be designed to support them later:

- Integration of additional sensors (EC, NPK, light intensity, rain, etc.)
- Crop disease detection using image-based AI models
- Automated irrigation control (relay-based)

- Advanced analytics dashboards for multi-year trends
- Multi-crop support for scaling the system commercially

6.6 Reusability Requirements

- REQ-REU-1: System modules (sensor reading module, recommendation engine, UI components) shall be designed as reusable components for future crops.
- REQ-REU-2: The AI layer, if added, shall be built as a separate microservice to allow reuse in other agricultural solutions.

Appendix A: Glossary

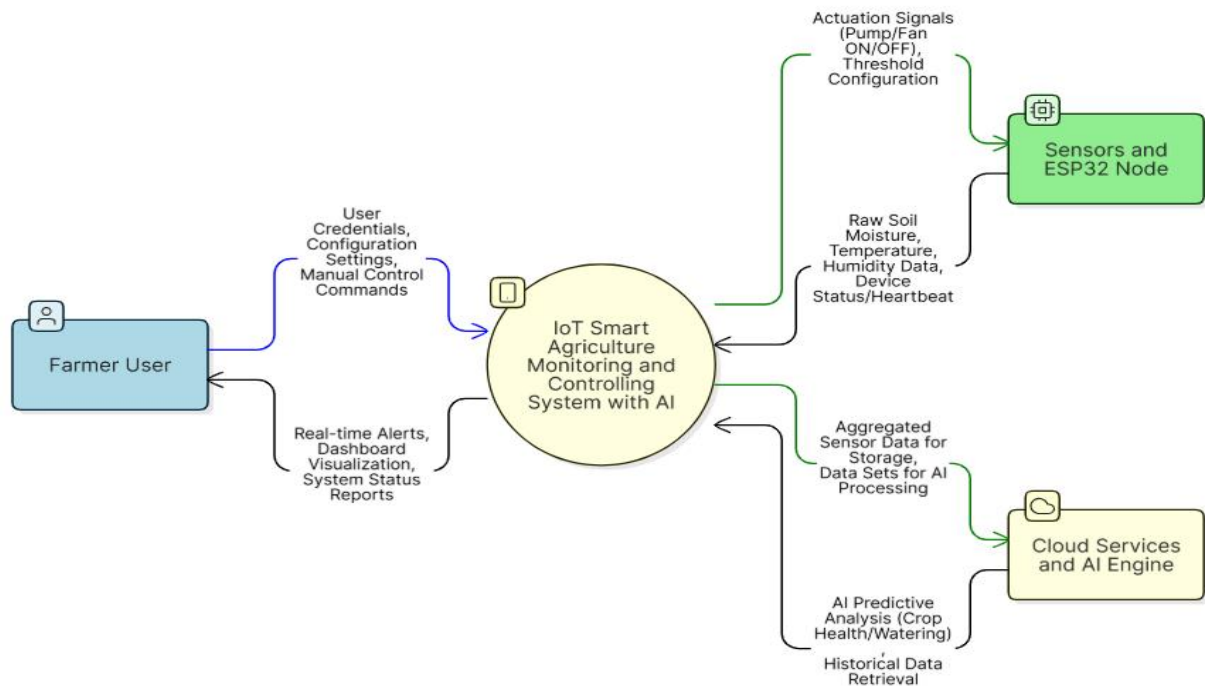
Term / Acronym	Definition
IoT (Internet of Things)	A network of physical devices embedded with sensors, software, and connectivity that enables data exchange.
ESP32	A low-cost WiFi + Bluetooth microcontroller used for IoT applications.
Sensor Node	A hardware unit consisting of ESP32 and connected sensors for collecting soil/environment data.
pH Sensor	Sensor that measures the acidity or alkalinity of the soil.
Soil Moisture Sensor	Sensor used to measure the amount of water present in the soil.
EC (Electrical Conductivity)	A measure indicating nutrient/salt concentration in soil; can be used to infer fertilizer needs.
Mobile Application (App)	Flutter-based interface used by farmers to view data, alerts, and recommendations.
Offline Mode	System functionality where the phone connects to ESP32 using Bluetooth when no internet is available.

Online Mode	Functionality where the system uses WiFi/cloud to provide enhanced recommendations and storage.
Optimum Conditions	Ideal soil/environment values (moisture, pH, EC, temperature, humidity) required for healthy tobacco growth.
Rule-Based Recommendation Engine	A logical comparison system that generates suggestions based on deviations from optimum values.
AI/ML (Artificial Intelligence / Machine Learning)	Algorithms that may analyze historical data to improve recommendations (optional depending on time/resources).
Cloud Backend	Server or online service used to store data, run algorithms, and sync with the mobile app.
Farmer (End User)	The person who uses the system to monitor crop conditions and receive recommendations.
Bluetooth Low Energy (BLE)	A low-power wireless communication method used for offline sync.
WiFi Communication	Wireless connection used for sending data to cloud services in online mode.
Recommendation	System-generated suggestion based on sensor data (e.g., "Irrigation needed").
Alert	Notification sent to the farmer when a parameter goes outside acceptable range.
Field Node	The installed hardware unit in the farm.
Data Sync	Transfer of collected data between ESP32 and mobile app (offline) or between app and cloud (online).

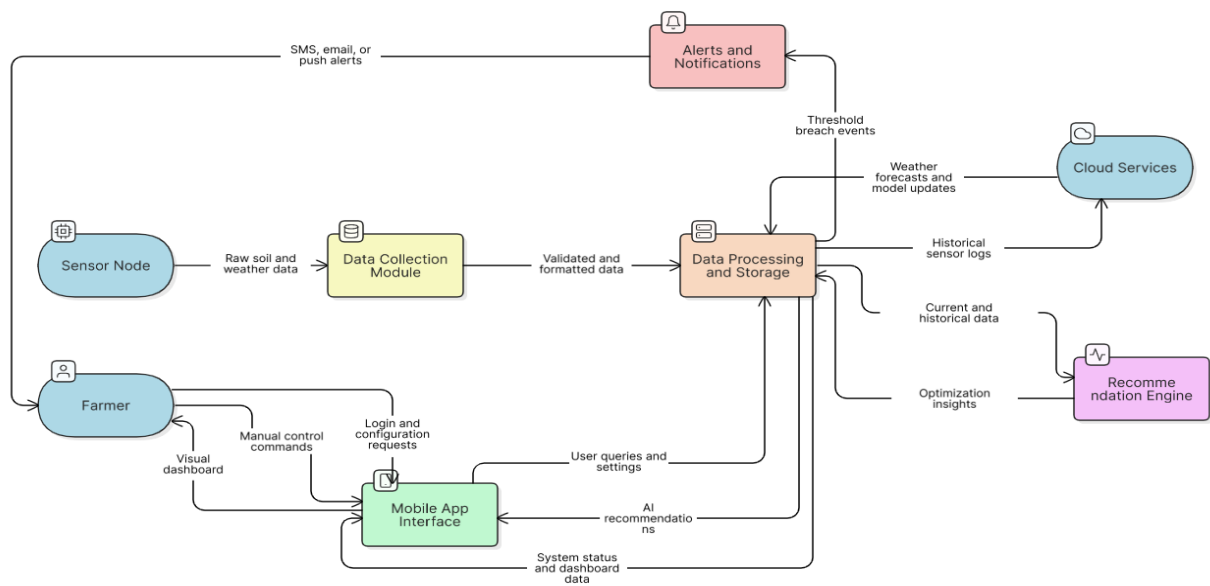
Appendix B: Analysis Models

This section contains the analysis models used to represent the structure and data flow of the IoT Smart Agriculture Monitoring and Controlling System with AI. These models help clarify how different components interact and how data moves through the system.

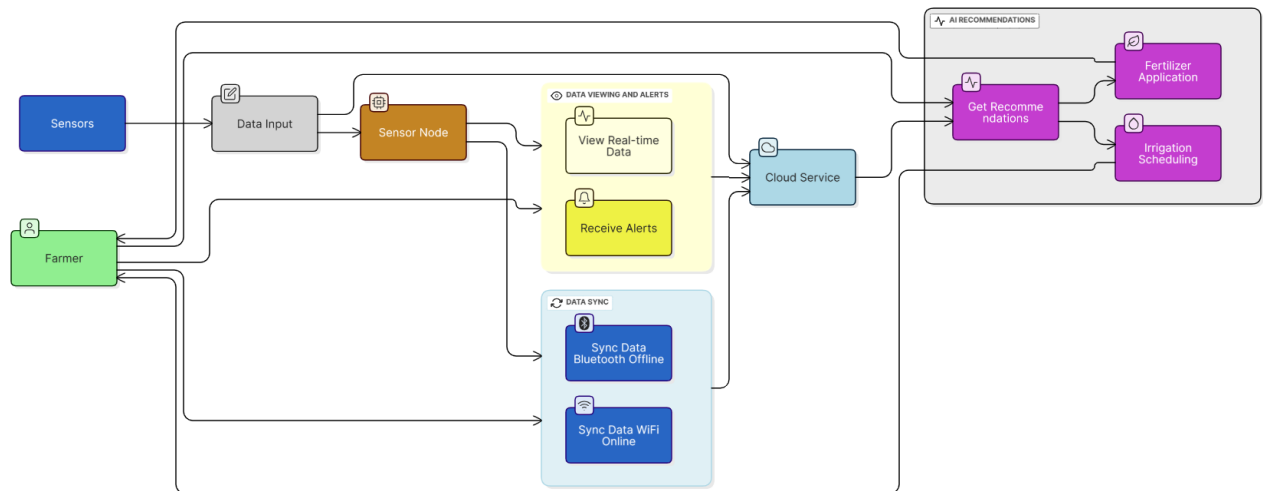
B.1 – Context Diagram (0-Level DFD)



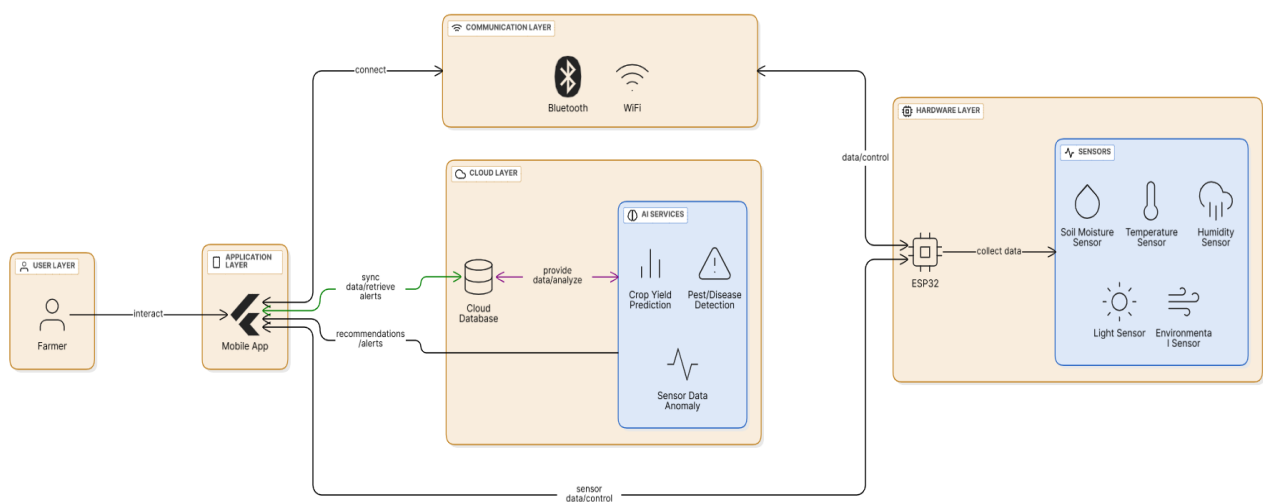
B.2 – Level-1 DFD (Decomposition of Main System)



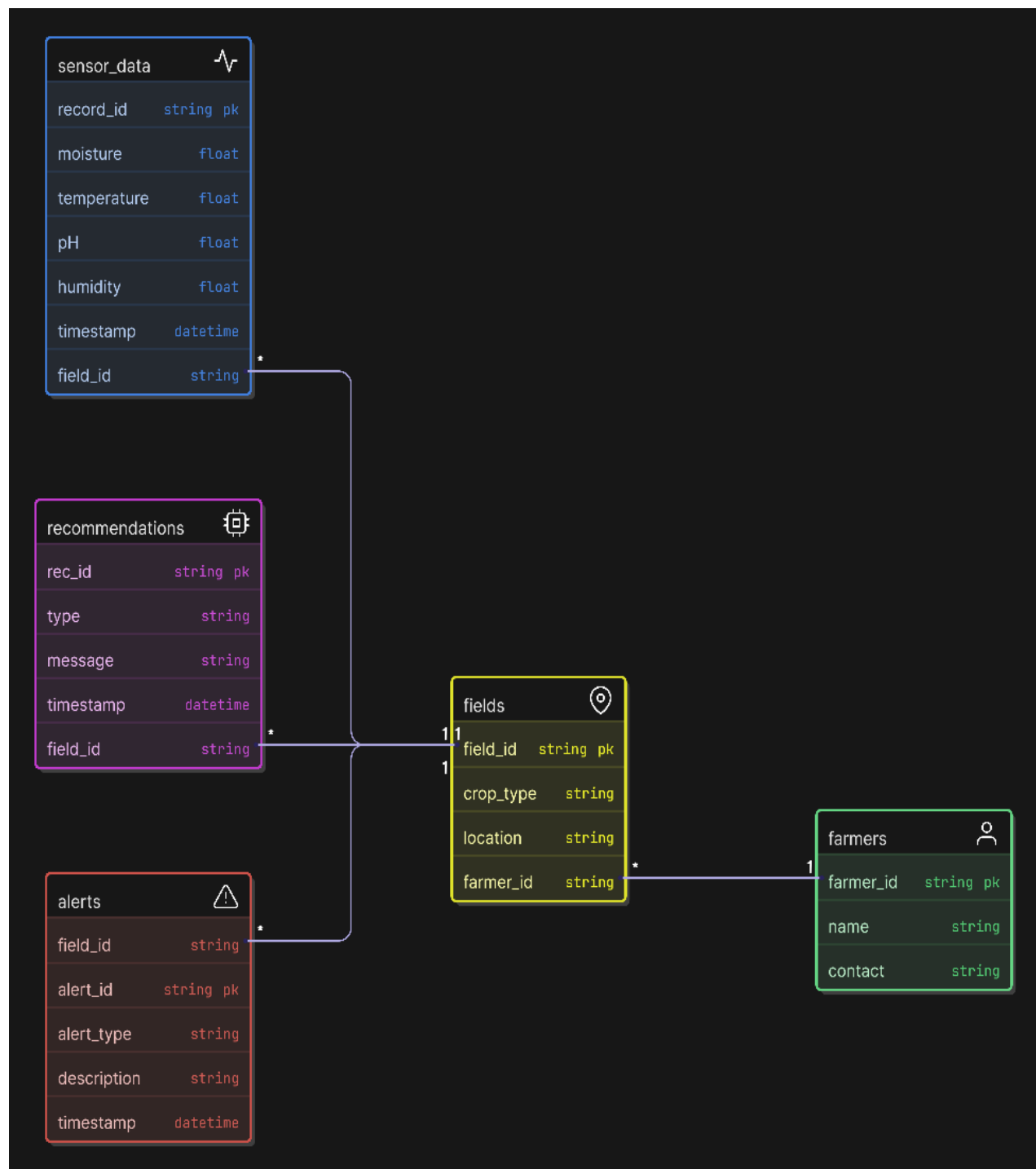
B.3 – Use Case Diagram



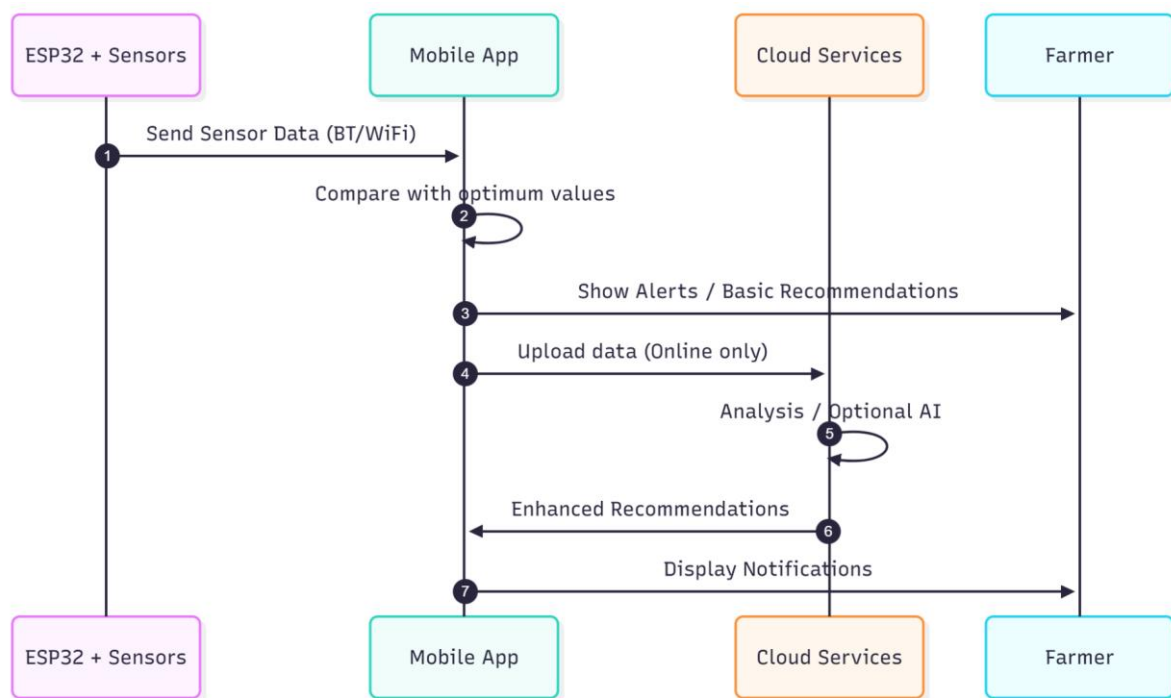
B.4 – System Architecture (Layered Architecture Diagram)



B.5 – Entity-Relationship (ER) Diagram (High Level)



B.6 – System Workflow Diagram



Appendix C: To Be Determined (TBD) List

This section lists all the requirements, decisions, and information that are not yet finalized and will be completed during later phases of the project. Each item has a unique identifier for tracking.

TBD-1: Final List of Sensors

The exact set of sensors (e.g., pH sensor model, EC sensor, light sensor) will be finalized after cost evaluation and availability in the local market.

TBD-2: Optimum Tobacco Crop Threshold Values

Exact optimum ranges for:

- Soil moisture
- Temperature
- Humidity

- pH

- EC

These will be finalized after consultations with agricultural experts or official sources.

TBD-3: Cloud Platform Selection

The choice among:

- Firebase
- Django REST API backend
- Other cloud service
remains pending and will be finalized after team evaluation.

TBD-4: Database Schema (Final Version)

A draft ERD has been created, but final attributes and tables will be confirmed during development.

TBD-5: Offline Storage Strategy

Exact format for offline data storage (e.g., local SQLite structure) is yet to be finalized.

TBD-6: Exact Bluetooth Sync Protocol

Final decision on:

- Data packet size
- Sync interval
- Error handling
will be determined during integration testing.

TBD-7: AI Model Inclusion

Whether AI-based fertilizer optimization or disease detection is included in FYP Phase-I or Phase-II depends on time/resources.

TBD-8: Data Collection Timeline

Exact schedule for local field data collection is not yet confirmed and depends on farm access and supervisor approval.

TBD-9: App UI Language Support

Whether multilingual support (Urdu/Pashto) will be included in the initial version or later updates.

TBD-10: Alert Types and Threshold Logic

The complete list of alerts (irrigation, fertilizer, environment risk) will be finalized during requirement refinement.